Relazione di Progetto: Network and Cloud Infrastructures

- **Autore:** Luca Esposito
- **Matricola:** DE9000021
- **Repository GitHub:** https://github.com/TheGiggi24/NCIS_unina/tree/main/2026/esposito_luca

1. Introduzione e Obiettivi del Progetto

Il presente project work si inserisce nell'ambito delle **Software-Defined Networking (SDN)**, un paradigma architetturale che disaccoppia il piano di controllo (*Control Plane*) dal piano di inoltro dei dati (*Data Plane*). L'obiettivo principale è la progettazione, l'implementazione e la validazione sperimentale di un controller SDN avanzato basato su **Ryu** e una topologia di rete virtualizzata emulata tramite **Mininet**.
Il sistema sviluppato integra tre pilastri fondamentali per le infrastrutture di rete moderne:

1. **Network e Service Slicing (QoS basata su priorità dei flussi):** Isolamento logico del traffico tra tenant differenti e priorità differenziate per tipologia di traffico (es. TCP/UDP a priorità 50, ICMP a priorità 20, traffico standard a 10).

2. **Mitigazione Dinamica degli Attacchi DoS (Denial of Service):** Un sistema di rilevamento a doppio strato (reattivo via Packet-In e statistico via polling dei flussi) combinato con un meccanismo di ban esponenziale basato su finestre temporali configurabili.

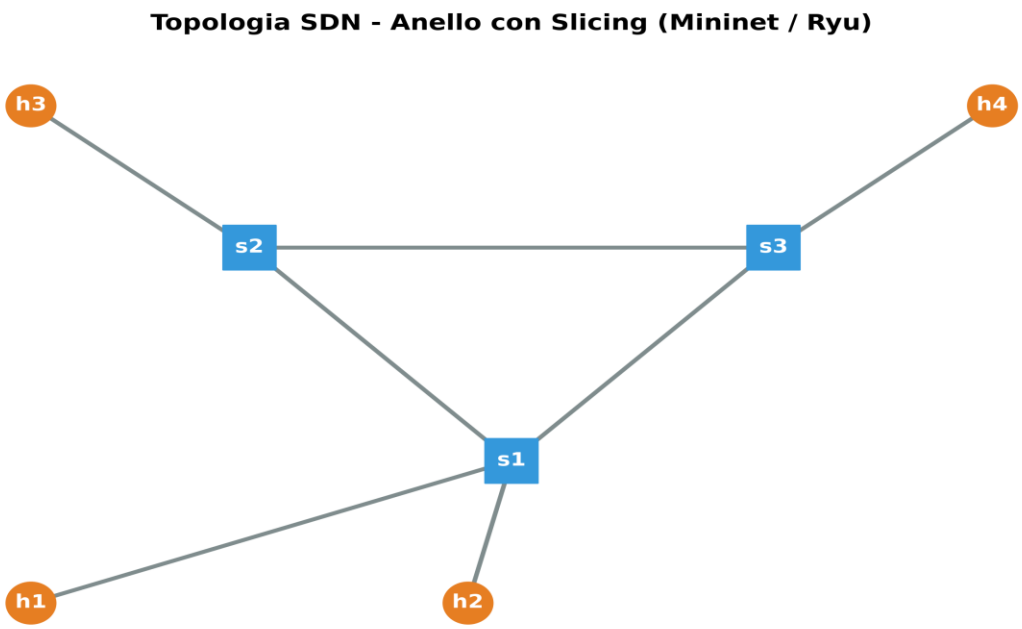
3. **Resilienza e Convergenza STP (Spanning Tree Protocol):** Gestione di topologie ad anello per prevenire i loop di layer 2 e garantire la continuità del servizio tramite failover automatico in caso di guasto fisico dei collegamenti.

2. Architettura della Rete e Topologia

La topologia di rete implementata (topology_slicing.py) è costituita da:

- **3 Switch OpenFlow (vSwitch -** s1**,** s2**,** s3**)** connessi in configurazione ad anello ridondante.

- **4 Host finali (**h1**,** h2**,** h3**,** h4**)** distribuiti tra i vari switch, suddivisi in due coppie di tenant isolati
    - *Tenant A (Slice A):* 10.0.0.1 (h1) e 10.0.0.3 (h3)
    - *Tenant B (Slice B):* 10.0.0.2 (h2) e 10.0.0.4 (h4)



La gestione dei loop di layer 2 sull'anello è delegata al modulo stplib integrato nel controller Ryu, il quale monitora lo stato delle porte e applica dinamicamente gli stati di sicurezza (BLOCKING, LISTENING, LEARNING, FORWARDING).

3. Moduli Logici e Implementazione del Controller (controller_slicing.py)

Il controller Ryu (FinalSTPController) gestisce in modo centralizzato le politiche di rete attraverso i seguenti componenti logici:

A. Network e Service Slicing (QoS)

Il controller analizza i pacchetti ARP e IPv4 in transito. Se viene rilevato un tentativo di comunicazione tra indirizzi appartenenti a slice differenti (es. da Slice A a Slice B), il flusso viene intercettato e bloccato preventivamente mediante l'installazione di una regola di scarto hardware (actions=[]) con priorità elevata (priority=80), registrando l'evento ([SLICING] Traffico bloccato tra tenant). Parallelamente, il traffico legittimo viene

classificato in base alla QoS: flussi TCP/UDP ricevono priorità alta (50), ICMP priorità media (20) e il traffico standard di default priorità bassa (10).

B. Sistema di Mitigazione DoS a Doppio Strato e Gestione dei Buffer

La sicurezza anti-DoS è strutturata su due livelli complementari:

1. **Controllo Reattivo sui Packet-In:** Monitoraggio in tempo reale dei pacchetti inviati da ciascun host sorgente entro una finestra temporale rigorosa (self.TIME_WINDOW = 1.0 secondo). Se il numero di pacchetti supera la soglia (self.RATE_LIMIT = 5), il controller attiva un ban reattivo.

2. **Monitoraggio Statistico Attivo:** Un thread in background (_monitor) interroga periodicamente gli switch tramite OFPFlowStatsRequest per tracciare i volumi di traffico a livello di data plane. Se un flusso supera la soglia statistica, scatta il ban di protezione ([DOS DA STATISTICHE]).

Gestione delle Code e Buffer Overflow: Gli switch utilizzano code di trasmissione (*Transmission Queues*) per gestire la concorrenza verso una stessa destinazione. In condizioni di sovraccarico o attacco flood (es. test a 75 o 400 pacchetti), i buffer vanno in overflow causando il drop dei pacchetti (*tail drop*), mitigato efficacemente dal blocco reattivo del controller.

C. Ban Esponenziale Reattivo

Quando un host viola la soglia di traffico, viene applicato un blocco hardware temporaneo mediante una regola OpenFlow con hard_timeout dinamico basato sul numero di infrazioni ripetute (Durata = BASE_BAN_TIME × $2^{(\text{infrazioni} - 1)}$). Ad esempio, la prima infrazione comporta l'isolamento immediato dell'host per 15 secondi, poi 30s per le infrazioni ripetute sotto carico elevato.

4. Validazione Sperimentale e Test di Carico

La validazione del sistema è stata condotta interamente in ambiente emulato (Mininet + Ryu), verificando gli scenari critici di seguito descritti:

1. **Test di Confine e Flood DoS:** Invio di flussi ravvicinati (es. h1 ping -c 75 -i 0.1 h3 con il 90.6% di perdita e h1 ping -c 400 -i 0.1 h3 con il 97.5% di perdita) per testare il superamento della soglia e il conseguente blocco reattivo del traffico malevolo.

2. **Test di Isolamento (Slicing):** Tentativi di comunicazione inter-tenant intercettati e bloccati istantaneamente ([SLICING] Traffico bloccato tra tenant).

3. **Test di Resilienza STP e Failover (Collasso Link):** Simulazione di un guasto fisico sulla topologia ad anello tramite il comando `link s1 s2 down` durante un ping continuo in background. L'STP ha rilevato istantaneamente il cambio di topologia, avviando il ricalcolo dei percorsi attraverso le transizioni di stato (BLOCKING -> LISTENING -> LEARNING -> FORWARDING), portando la rete a convergenza e ripristinando la connettività sul percorso alternativo.

5. Output dei Terminali e Log Sperimentali (DoS, Slicing e STP)

Di seguito vengono riportati i log effettivi catturati durante le sessioni di test in Mininet e sul controller Ryu, evidenziando il rilevamento del DoS, l'isolamento dello slicing e il ricalcolo dell'anello dovuto al collasso del link.

Log di Mininet (Sessione di Test):

```
Plaintext
root@057a396a9855:/# mn --custom /workspace/topology_slicing.py --topo slicingtopo --switch
ovs,datapath=user --controller=remote,ip=127.0.0.1,port=6633 --mac
*** Creating network
*** Adding controller
*** Adding hosts: h1 h2 h3 h4
*** Adding switches: s1 s2 s3
*** Adding links: (h1, s1) (h2, s1) (h3, s2) (h4, s3) (s1, s2) (s1, s3) (s2, s3)
*** Starting CLI:
mininet> h1 ping -c 6 -i 0.1 h4
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
--- 10.0.0.4 ping statistics ---
6 packets transmitted, 0 received, +6 errors, 100% packet loss, time 546ms

mininet> h1 ping -c 75 -i 0.1 h3
--- 10.0.0.3 ping statistics ---
75 packets transmitted, 7 received, 90.6667% packet loss, time 7806ms

mininet> h1 ping -c 400 -i 0.1 h3
--- 10.0.0.3 ping statistics ---
400 packets transmitted, 10 received, 97.5% packet loss, time 42731ms

mininet> h1 ping h3 &
mininet> link s1 s2 down
mininet> link s1 s2 up
```

```
mininet> pkill -9 ping
```

Log corrispondenti nel terminale del Controller Ryu (Eventi di Sicurezza e Convergenza STP):

```
Plaintext
[STP][INFO] dpid=0000000000000001: Join as stp bridge.
[STP] Switch 1, Porta 1 -> BLOCKING (Bloccata)
[STP] Cambio topologia rilevato! Ricalcolo i percorsi...
[STP] Switch 1, Porta 2 -> FORWARDING (Attiva)

>>> [DOS DA PACKET_IN] Host 00:00:00:00:00:01 bloccato per 15s! (Infrazione #1) <<<
[SLICING] Traffico bloccato tra tenant: 10.0.0.1 -> 10.0.0.4
>>> [DOS DA STATISTICHE] Host 00:00:00:00:00:03 bloccato per 15s! (Infrazione #1) <<<
>>> [DOS DA STATISTICHE] Host 00:00:00:00:00:01 bloccato per 30s! (Infrazione #2) <<<

[STP][INFO] dpid=0000000000000001: [port=1] Link down.
[STP] Switch 1, Porta 1 -> BLOCKING (Bloccata)
[STP] Cambio topologia rilevato! Ricalcolo i percorsi...
[STP] Switch 2, Porta 1 -> NON_DESIGNATED_PORT / BLOCK
[STP][INFO] dpid=0000000000000001: [port=1] Link up.
[STP] Cambio topologia rilevato! Ricalcolo i percorsi... Rete a convergenza.
```

6. Dump delle Regole OpenFlow dello Switch

Output del comando `ovs-ofctl dump-flows` eseguito sullo switch `s1`, che evidenzia la corretta installazione delle regole di priorità (Slicing, DoS e STP):

Plaintext

```
cookie=0x0, duration=2146.823s, table=0, n_packets=1078, n_bytes=64680,
priority=65535,dl_dst=01:80:c2:00:00:00 actions=CONTROLLER:65535
 cookie=0x0, duration=1679.624s, table=0, n_packets=2, n_bytes=84,
priority=80,arp,arp_spa=10.0.0.1,arp_tpa=10.0.0.4 actions=drop
 cookie=0x0, duration=2047.917s, table=0, n_packets=376, n_bytes=35504, priority=10,in_port="s1-
eth2",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth3"
 cookie=0x0, duration=2047.916s, table=0, n_packets=1385, n_bytes=134386,
priority=20,in_port="s1-eth3",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03
actions=output:"s1-eth2"
 cookie=0x0, duration=2146.826s, table=0, n_packets=19, n_bytes=1218, priority=0
actions=CONTROLLER:65535
```